

PROJET ASSEMBLEUR

Licence d'informatique 2024 -2025

Auteurs :

- BENVENISTE Alexandre
- NAVARRO Antony

Introduction :

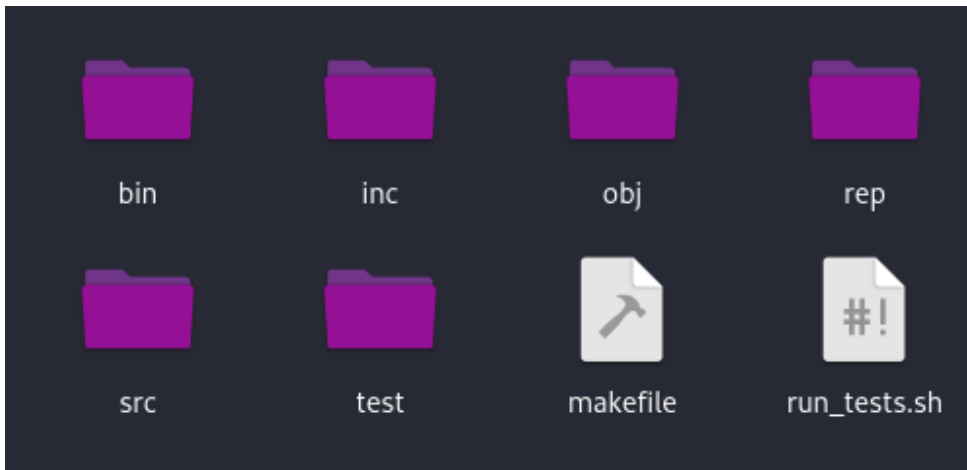
Ce projet consiste à développer un assembleur qui prend d'abord un fichier "tpc", un sous-ensemble du langage C, puis fait une analyse syntaxique en utilisant les outils ****Flex**** et ****Bison****. Ensuite, quand l'analyse est réussie, on vérifie si la sémantique en C est respectée. Enfin, quand la sémantique est validée, on fait la traduction du programme en assembleur. L'objectif du programme est de faire la traduction d'un fichier "tcp" en un fichier assembleur.

Le projet inclut également un script d'automatisation pour exécuter des tests sur des programmes corrects et incorrects, et retourne des rapports d'analyse et de sémantique.

Objectifs :

- Vérification des fichiers "tcp" : Détecter les erreurs dans les programmes.
- Construction de l'arbre et de la table des symboles : Représenter la structure des programmes corrects sous forme d'un arbre syntaxique abstrait et d'une table des symboles pour la traduction.
- Automatisation des tests : Fournir des outils pour tester et valider le fonctionnement de l'analyseur et de la sémantique.

Structure du projet :



- bin : le fichier exécutable tpcc
- obj : les fichiers .o
- src : codegen.c, compilateur.c, semantique.c, tabsymbole.c, tpc.lex, tpc-2024-2025.y et tree.c
- inc : codegen.h, compilateur.h, semantique.h, tabsymbole.h et tree.h
- rep: le rapport du projet
- test :
 - good : tous les fichiers tests corrects (code de retour 0)
 - syn-err : tous les fichiers tests de syntaxes incorrects (code de retour 1)
 - sem-err : tous les fichiers tests de sémantiques incorrects (code de retour 2)
 - warn : tous les fichiers tests avec des warnings (code de retour 0 avec warning)
- makefile
- run_tests.sh : le fichier script.

Fonctionnalités des Modules :

- tpc-2024-2025.y/tpc.lex : Permet de faire l'analyse syntaxique d'un fichier tcp
- tree.c/h : Permet de créer un arbre abstrait du fichier tcp quand l'analyse syntaxique est réussie. Elle renvoie un code d'erreur 1 si une erreur de syntaxe est détectée.
- tabsymbole.c/h : Permet de générer la table des symboles de chaque fonction et la table des symboles globale pour avoir toutes les fonctions et les variables globales
- semantique.c/h : Permet de faire la vérification de la sémantique d'un fichier tpc en utilisant la table des symboles et l'arbre abstrait de celui-ci. Elle renvoie un code d'erreur 2 si une erreur de sémantique est détectée et des warnings

- [codegen.c/h](#) : Permet de créer un fichier assembleur “_anonymous.asm” à partir de l’arbre abstrait et des tables de symboles obtenues
- [compilateur.c/h](#) : Permet d'exécuter les programmes codegen, sémantique et tabsymbole. Il vérifie si la sémantique est correcte avant de créer le fichier assembleur

Conception :

- [Analyse lexicale et syntaxique :](#)

L’analyse lexicale est réalisée à l’aide de ****Flex**** et l’analyse syntaxique est définie à l’aide de ****Bison****. La grammaire dans notre projet respecte celle donnée sur elearning.

- [Arbre syntaxique abstrait :](#)

L’AST est construit à l’aide de nœuds représentant les éléments syntaxiques du programme

- [Tables des symboles :](#)

La première table des symboles construite est la table globale, qui recense toutes les fonctions et variables globales déclarées dans le programme, avec leurs types, portées et attributs (statique, local, etc.). Ensuite, pour chaque fonction, nous générons une table des symboles locale qui contient les paramètres et variables locales, ainsi que leurs types et offsets sur la pile. Lors de l’affichage, nous imprimons d’abord la table globale, puis, pour chaque fonction, sa table de symboles locale, facilitant ainsi la vérification de la portée et de la résolution des identificateurs

- [Analyse sémantique :](#)

Le module sémantique vérifie la cohérence du programme après l’analyse syntaxique : il s’assure que tous les identifiants sont déclarés avant utilisation, que les types des variables et des expressions sont compatibles lors des affectations et des retours de fonction, que les appels de fonctions respectent le nombre et le type des arguments attendus, et qu’une fonction main de type int existe. Il signale les erreurs sémantiques (redéclarations, types incompatibles, identifiants non déclarés) et émet des avertissements pour les conversions risquées, garantissant ainsi la validité logique du code source avant la génération du code machine.

- [Génération de code NASM :](#)

La fonction generateNASM parcourt l’AST en commençant par un court en-tête NASM (sections .data et .text, déclaration des variables globales et de l’entrée), puis traduit :

- Prologue/épilogue des fonctions : gestion de la pile, alignement, stockage et restitution des paramètres.
- Structures de contrôle : boucles while et instructions if sont réalisées avec étiquettes uniques et sauts conditionnels.

- Expressions et affectations : opérations arithmétiques et logiques posant les résultats sur la pile, comparaisons évaluées en booléen, et mouvements en mémoire.
- Appels de fonctions : évaluation des arguments, empilement, appel et empilement du retour.

Les variables locales sont placées dans .bss et initialisées à zéro, et les fonctions (getchar, getint, putchar, putint) sont directement écrites en assembleur pour être appelées depuis le code généré.

- Gestion des erreurs :

L'analyseur syntaxique détecte et signale les erreurs lexicales et syntaxiques et le compilateur détecte les erreurs de sémantiques ainsi que des warning en affichant le numéro de ligne et une description de l'erreur. Le programme peut aussi renvoyer une erreur s'il y a un problème interne (table des symboles pleines ou problème d'allocation de mémoire).

Tests :

Organisation des tests :

- tests du dossier good : le programme ne doit pas envoyer une erreur.
- tests du dossier syn-err : le programme doit détecter une erreur de syntaxe
- tests du dossier sem-err : le programme doit détecter une erreur de sémantique
- tests du dossier warn : le programme doit renvoyer un warning mais pas d'erreur

Script d'exécution des tests :

Le script run_tests.sh automatise l'exécution des tests et génère un rapport contenant les résultats et un score global.

```
alex@PCOVER9000:~/ProjetCompile$ ./run_tests.sh syn-err

=== TESTS DANS syn-err (test/syn-err) ===
Test test/syn-err/big.tpc :[FAIL] (Code obtenue : 0, Code attendu : 1)
---- STDERR ----
Code erreur : 0
-----
Test test/syn-err/big_syserr.tpc :[OK]
Test test/syn-err/eq_vars.tpc :[OK]
Test test/syn-err/global_static.tpc :[OK]
Test test/syn-err/no_fun.tpc :[OK]
Test test/syn-err/no_parameter.tpc :[OK]
Test test/syn-err/no_return.tpc :[FAIL] (Code obtenue : 2, Code attendu : 1)
---- STDERR ----
error (line 2): function 'foo' missing return
error (line 5): function returning 'char' must return a value
Compilation failed due to semantic errors.
Code erreur : 2
-----
Test test/syn-err/no_type.tpc :[OK]
Test test/syn-err/printf.tpc :[OK]
==> Résultat syn-err : 7 / 9

=== RÉCAPITULATIF GLOBAL ===
Tests syn-err      : 7 / 9
Score total        : 7 / 9
```

Compilation du programme :

Pour compiler le programme, on se positionne dans le répertoire du projet et utiliser la commande:
- “make”

```
alex@PCOVER9000:~/ProjetCompile$ make
mkdir -p obj
mkdir -p bin
bison -d -o obj/y.tab.c src/tpc-2024-2025.y
flex -o obj/lex.yy.c src/tpc.lex
gcc -Wall -g -Iinc -c obj/lex.yy.c -o obj/lex.yy.o
gcc -Wall -g -Iinc -c obj/y.tab.c -o obj/y.tab.o
gcc -Wall -g -Iinc -c src/tree.c -o obj/tree.o
gcc -Wall -g -Iinc -c src/compilateur.c -o obj/compilateur.o
gcc -Wall -g -Iinc -c src/tabsymbole.c -o obj/tabsymbole.o
gcc -Wall -g -Iinc -c src/semantique.c -o obj/semantique.o
gcc -Wall -g -Iinc -c src/codegen.c -o obj/codegen.o
gcc -Wall -g -Iinc obj/lex.yy.o obj/y.tab.o obj/tree.o obj/compilateur.o obj/tabsymbole.o obj/semantique.o obj/codegen.o -o bin/tpcc
```

Lancement des testes :

Pour utiliser le script qui évalue nos fichiers de tests, on utilise la commande : - “make test”

```
alex@PCOVER9000:~/ProjetCompile$ make test
./run_tests.sh

=== TESTS DANS good (test/good) ===
Test test/good/big.tpc :[OK]
Test test/good/bool.tpc :[OK]
Test test/good/example.tpc :[OK]
Test test/good/functions.tpc :[OK]
Test test/good/gen-code-types.tpc :[OK]
Test test/good/global.tpc :[OK]
Test test/good/if.tpc :[OK]
Test test/good/not.tpc :[OK]
Test test/good/operation.tpc :[OK]
Test test/good/order.tpc :[OK]
Test test/good/return_types.tpc :[OK]
Test test/good/static.tpc :[OK]
Test test/good/vars.tpc :[OK]
Test test/good/while.tpc :[OK]
==> Résultat good : 14 / 14

=== TESTS DANS sem-err (test/sem-err) ===
Test test/sem-err/no_return.tpc :[OK]
Test test/sem-err/not_declare.tpc :[OK]
Test test/sem-err/param_local_conflict.tpc :[OK]
Test test/sem-err/redecl_fun.tpc :[OK]
Test test/sem-err/redecl_var.tpc :[OK]
Test test/sem-err/return_wrong_type.tpc :[OK]
Test test/sem-err/undeclared_fun.tpc :[OK]
Test test/sem-err/undeclared_var.tpc :[OK]
Test test/sem-err/var_fun_conflict.tpc :[OK]
==> Résultat sem-err : 9 / 9

=== TESTS DANS syn-err (test/syn-err) ===
Test test/syn-err/big_syserr.tpc :[OK]
Test test/syn-err/eq_vars.tpc :[OK]
Test test/syn-err/global_static.tpc :[OK]
Test test/syn-err/no_fun.tpc :[OK]
Test test/syn-err/no_parameter.tpc :[OK]
Test test/syn-err/no_type.tpc :[OK]
Test test/syn-err/printf.tpc :[OK]
==> Résultat syn-err : 7 / 7

=== TESTS DANS warn (test/warn) ===
Test test/warn/stack.tpc :[OK]
Test test/warn/warning.tpc :[OK]
Test test/warn/warning2.tpc :[OK]
==> Résultat warn : 3 / 3

=== RÉCAPITULATIF GLOBAL ===
Tests good      : 14 / 14
Tests syn-err    : 7 / 7
Tests sem-err    : 9 / 9
Tests warn       : 3 / 3
Score total      : 33 / 33
```

Voici le bilan du script, on peut voir qu’il y a 5 blocs.

Bloc des tests de programmes :

Affichent tous les tests selon le dossier et montre pour chaque test si celui-ci est valide.

[OK] -> le test est passé (renvoie le code d'erreur attendu)

[FAIL] -> le test a échoué (le code d'erreur reçu est différent de celui attendu)

Si le test échoue, le testeur va afficher la sortie d'erreur du programme testé

Bloc récapitulatif :

Affiche le score de chaque dossier de tests et le score total.

Exécution du programme :

Pour utiliser le programme, on utilise la commande :

`./bin/tpcc [option] mon_fichier.tpc` ou `./bin/tpcc [option] < mon_fichier.tpc`

Options :

→ `-t, --tree` : permet d'afficher l'arbre abstrait du programme.

```
alex@PCOVER9000:~/ProjetCompile$ ./bin/tpcc -t test/good/global.tpc
Analyse réussie.
Program
├── Global
│   ├── int
│   │   └── Ident (FOO)
│   ├── char
│   │   └── Ident (BAR)
│   ├── int
│   │   ├── Ident (MIN)
│   │   └── Ident (MAX)
│   └── char
│       ├── Ident (FILE)
│       └── Ident (TEXT)
└── Functions
    └── Function
        ├── Head
        │   ├── int
        │   ├── main
        │   └── Parameter
        │       └── void
        └── Body
            ├── Vars
            └── Suite_instr
                └── Return
                    └── 0

Code erreur : 0
```

→ -s, --syntabs : permet d'afficher toutes les tables des symboles.

```
alex@PCOVER9000:~/ProjetCompile$ ./bin/tpcc -s test/good/global.tpc
Analyse réussie.
Nom: getchar, Type: char, Portée: Global, Statique: Non, Paramètre: Non, Adresse: 0
Nom: putchar, Type: void, Portée: Global, Statique: Non, Paramètre: Non, Adresse: 0
Nom: getint, Type: int, Portée: Global, Statique: Non, Paramètre: Non, Adresse: 0
Nom: putint, Type: void, Portée: Global, Statique: Non, Paramètre: Non, Adresse: 0
Nom: FOO, Type: int, Portée: Global, Statique: Non, Paramètre: Non, Adresse: 0
Nom: BAR, Type: char, Portée: Global, Statique: Non, Paramètre: Non, Adresse: 8
Nom: MIN, Type: int, Portée: Global, Statique: Non, Paramètre: Non, Adresse: 9
Nom: MAX, Type: int, Portée: Global, Statique: Non, Paramètre: Non, Adresse: 17
Nom: FILE, Type: char, Portée: Global, Statique: Non, Paramètre: Non, Adresse: 25
Nom: TEXT, Type: char, Portée: Global, Statique: Non, Paramètre: Non, Adresse: 26
Nom: main, Type: int, Portée: Global, Statique: Non, Paramètre: Non, Adresse: 0

Table des symboles de la fonction : main
Code erreur : 0
```

→ -h, --help : permet d'afficher le manuel d'utilisation du programme.

```
alex@PCOVER9000:~/ProjetCompile$ ./bin/tpcc -h
Commande :
  ./bin/tpcc [OPTIONS] [FICHIER]
Options :
  -t, --tree      Affiche l'arbre abstrait du fichier
  -h, --help      Affiche cette aide
  -s, --syntabs   Affiche toutes les tables de symboles
```

Valeurs de retour:

- 0 : succès (avec ou sans warnings).
- 1 : erreur lexicale ou syntaxique.
- 2 : erreur sémantique.
- >=3 : autres erreurs (ligne de commande, mémoire, non implémenté...).

Compilation du code nasm et exécution :

Exécutez la commande : `nasm -felf64 _anonymous.asm && ld _anonymous.o -o prog && ./prog`

Difficultés rencontrées:

- Écrire du code NASM qui n'interfère pas avec les portions déjà générées, sans casser la logique existante.
- Mise en place de la table des symboles, beaucoup de temps passé à trouver une implémentation correcte et fiable.
- Difficultés à obtenir des points sur le banc de tests NASM, malgré des cas de test assez complets et complexes.